МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине

«Проектирование вычислительных систем»

Вариант: 5

Работу выполнили:
Студенты группы P3411
Болорболд Аригуун
Кравцов Кирилл Денисович

Преподаватель:
Пинкевич Василий Юрьевич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 5
   2.1. Распиновка 5
   2.2. Конфигурация частоты ядра 5
   2.3. Блок-схема: 6
   2.4. Описание работы алгоритма 6
   2.5. Код драйвера 6
3. Вывод 13

1. Текст задания

Разработать программу, которая использует таймеры для управления яркостью светодиодов и излучателем звука (по прерыванию или с использованием аппаратных каналов). Блокирующее ожидание (функция HAL_Delay()) в программе использоваться не должно.

Стенд должен поддерживать связь с компьютером по UART и выполнять указанные действия в качестве реакции на нажатие кнопок на клавиатуре компьютера. В данной лабораторной работе каждая нажатая кнопка (символ, отправленный с компьютера на стенд) обрабатываются отдельно, ожидание ввода полной строки не требуется.

Для работы с UART на стенде можно использован один из двух вариантов драйвера (по прерыванию и по опросу) на выбор исполнителя. Поддержка двух вариантов не требуется.

Частота синхросигнала процессорного ядра и сигнала ШИМ для управления яркостью светодиодов (если используется) должны соответствовать указанным в варианте задания.

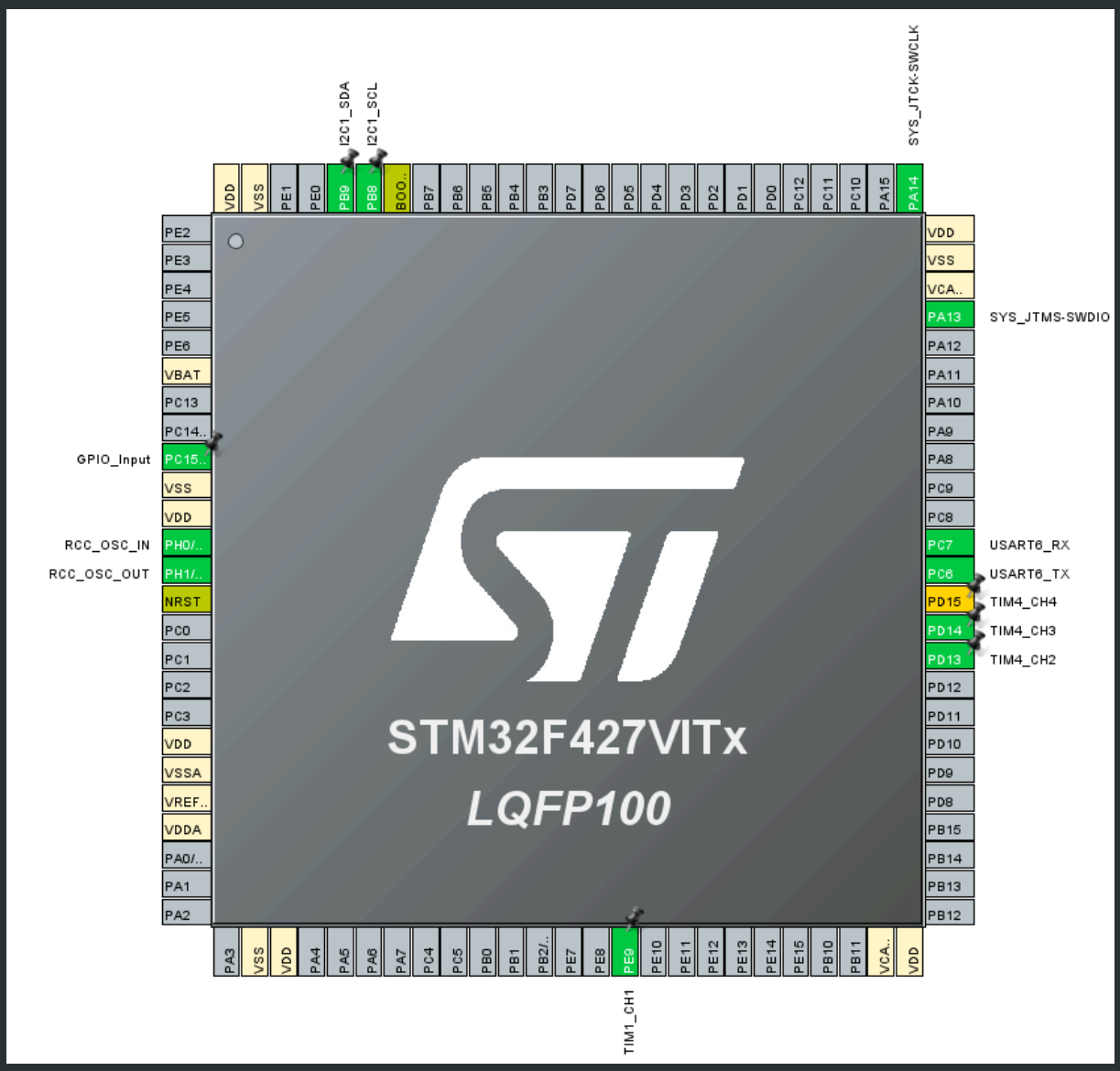
Реализовать музыкальную ритм-игру. С помощью звукоизлучателя воспроизводится последовательно, состоящая из звуков разной частоты («мелодия»). Каждый звук сопровождается зажиганием светодиода определенного цвета и с определенной яркостью (регулируется коэффициентом заполнения). Должно существовать взаимно однозначное соответствие между частотой звука и цветом/яркостью светодиода. Во время каждого звука/импульса светодиода игрок должен ввести символ, соответствующий текущей частоте звука или цвету/яркости. Чем больше звуков будет «угадано» правильно и на большей скорости игры, тем больше очков заработает игрок (система начисления очков – на усмотрение исполнителей). Всего необходимо предусмотреть девять видов импульсов: зеленый, желтый и красный на 20 %, 50 % и 100 % яркости. К ним следует подобрать звуки произвольных частот, легко отличимых одна от другой на слух. Предусмотреть одну стандартную последовательность импульсов длительностью не менее 20-ти элементов (простейший вариант – циклический перебор девяти импульсов). Когда последовательность заканчивается или досрочно останавливается игроком, в UART выводится количество набранных очков и «трассировка» нажатий, где отмечены правильные и неправильные нажатия. Отсутствие нажатия в течение импульса должно считаться неправильным нажатием. Действия стенда при получении символов от компьютера:

Символ	Действие
«1» – «9»	Символы, которые надо вводить в течение игры во время соответствующих импульсов. Когда игра не запущена, при вводе данных символов соответствующие импульсы воспроизводятся на светодиодах и излучателе звука, а в UART выводится описание импульса, которому соответствует символ (цвет/яркость и частота звука).
«+»	Циклически переключать скорость игры (длительность каждого импульса): малая, средняя, быстрая. Необходимо подобрать длительности импульсов так, чтобы на малой скорости было комфортно набирать очки, а максимальная скорость не была «непроходимой».
«a»	Циклически переключать режим воспроизведения последовательности: светодиоды и звук, только светодиоды, только звук.
«Enter»	Запустить или досрочно остановить игру. При запуске должна выдерживаться пауза в три секунды после ввода символа, чтобы игрок успел подготовиться.

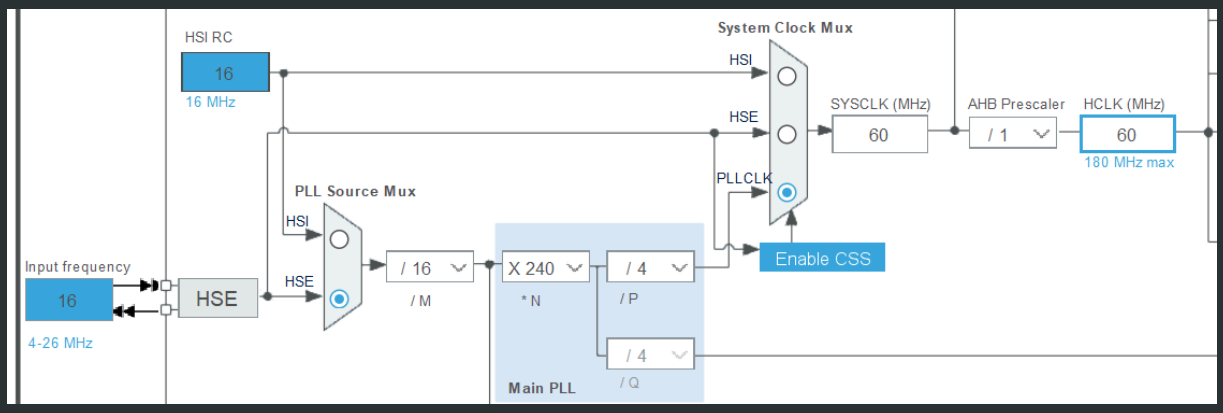
После ввода каждого символа в UART должно выводиться сообщение о том, какой импульс выбран или какой режим игры установлен. Частота процессорного ядра – 60 МГц, частота ШИМ-сигнала для светодиодов – 500 Гц

2. Выполнение

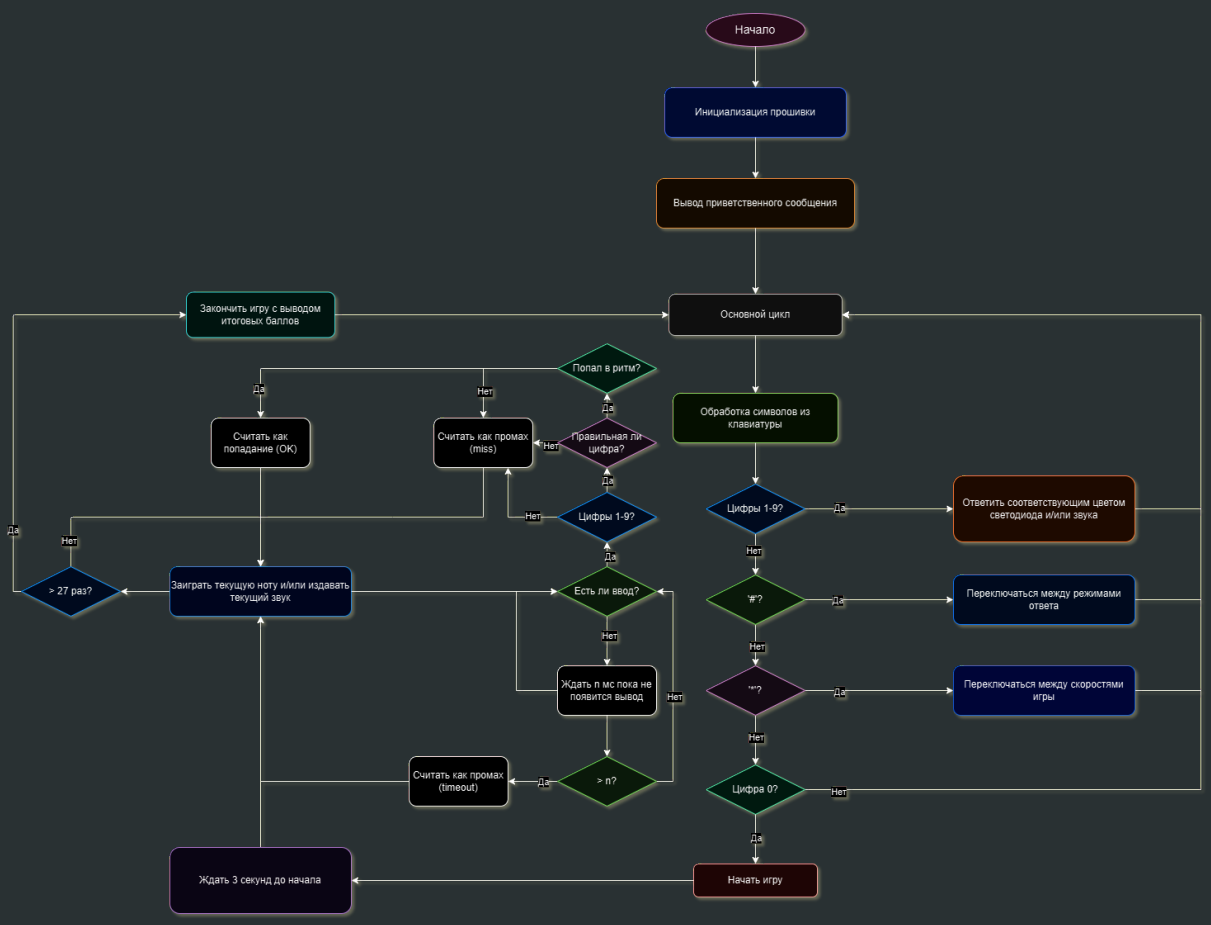
2.1. Распиновка



2.2. Конфигурация частоты ядра



2.3. Блок-схема:



2.4. Описание работы алгоритма

Надо в терминале выбрать правильный порт и установить соответствующую скорость передачи данных. После этого появится сообщение на экране, после чего можно будет тестировать код через клавиатуру устройства (не стенда). Если встретится цифр от 1 до 9, то стенд будет мигать соответствующим цветом и издавать звук. Если встретится символ «а», то изменится режим ответа на цифр — либо только светодиод, либо только звук, либо оба. Если пользователь (дальше «игрок») нажмет на «Enter», то начнется последовательность, где «игроку» надо будет в точь нажать на цифры, иначе это считается как промах, а если вовсе не нажмет, то после определенной времени будет указываться таймаут. В итоге программа считает, сколько раз игрок попал в ритм. При ввода символа плюса переключается скорость игры: 1с, 600мс, 300мс.

2.5. Код драйвера

```
extern TIM_HandleTypeDef htim1;
extern TIM_HandleTypeDef htim4;
extern TIM_HandleTypeDef htim6;
extern UART_HandleTypeDef huart6;
```

```
static Speed_t current_speed = SPEED_SLOW;
static PlayMode_t play_mode = MODE_BOTH;
static GameState_t game_state = GAME_IDLE;

static uint8_t sequence[SEQUENCE_LENGTH];
static uint8_t sequence_pos = 0;
static uint32_t score = 0;
static uint8_t mistakes = 0;
static uint8_t input_received = 0;
static char trace[SEQUENCE_LENGTH];

static uint32_t countdown_start = 0;
static uint32_t pulse_start = 0;

uint8_t uart_rx_buffer[128];
volatile uint16_t uart_rx_head = 0;
volatile uint16_t uart_rx_tail = 0;
uint8_t uart_rx_byte;

volatile uint32_t uart_tx_head = 0;
volatile uint32_t uart_tx_tail = 0;
uint8_t uart_tx_buffer[256];
volatile uint8_t uart_tx_busy = 0;

static void SetPWM(uint8_t green, uint8_t yellow, uint8_t red) {
    if (play_mode == MODE_SOUND_ONLY) {
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, 0);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, 0);
        return;
    }

    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, (green * 2000) /
100);
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, (yellow * 2000) /
100);
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, (red * 2000) /
100);
}

static void SetBuzzer(uint16_t freq) {
    if (play_mode == MODE_LED_ONLY || freq == 0) {
        HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
        return;
    }
```

```
    uint32_t timer_clock = 144000000;
    uint16_t prescaler = (timer_clock / 1000000) - 1;
    uint32_t arr = (1000000 / freq) - 1;

    if (arr > 65535) arr = 65535;
    if (arr < 10) arr = 10;

    char debug_buf[64];
    snprintf(debug_buf, sizeof(debug_buf), "[PSC=%u, ARR=%lu,
CCR=%lu] ", prescaler, arr, arr/2);
    UART_SendString(debug_buf);

    HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);

    htim1.Instance->PSC = prescaler;
    htim1.Instance->ARR = arr;
    htim1.Instance->CCR1 = arr / 2;
    htim1.Instance->CNT = 0;

    htim1.Instance->EGR = TIM_EGR_UG;

    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
}

static void StopBuzzer(void) {
    HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
}

static void PlayPulse(uint8_t pulse_idx) {
    const Pulse_t* p = &PULSES[pulse_idx];
    SetPWM(p->green, p->yellow, p->red);
    SetBuzzer(p->freq);
}

static void StopPulse(void) {
    SetPWM(0, 0, 0);
    StopBuzzer();
}

static void GenerateSequence(void) {
    for (int i = 0; i < SEQUENCE_LENGTH; i++) {
        sequence[i] = i % NUM_PULSES;
    }
}
```

```c
static void StartGame(void) {
    game_state = GAME_COUNTDOWN;
    countdown_start = HAL_GetTick();
    score = 0;
    mistakes = 0;
    sequence_pos = 0;
    memset(trace, 0, sizeof(trace));
    GenerateSequence();

    UART_SendString("\r\n=== GAME STARTING IN 3 SECONDS ===\r\n");
}

static void StopGame(void) {
    StopPulse();
    HAL_TIM_Base_Stop_IT(&htim6);
    game_state = GAME_FINISHED;
    PrintGameStats();
}

static void PrintGameStats(void) {
    char buf[256];
    snprintf(buf, sizeof(buf),
             "\r\n=== GAME FINISHED ===\r\n"
             "Score: %lu\r\n"
             "Correct: %d / %d\r\n"
             "Trace: ",
             score, SEQUENCE_LENGTH - mistakes, SEQUENCE_LENGTH);
    UART_SendString(buf);

    for (int i = 0; i < SEQUENCE_LENGTH; i++) {
        if (trace[i] == '+') {
            UART_SendString("[OK]");
        } else if (trace[i] == '-') {
            UART_SendString("[MISS]");
        } else {
            UART_SendString("[TIMEOUT]");
        }
    }
    UART_SendString("\r\n");
    game_state = GAME_IDLE;
}

static void ProcessInput(char c) {
    if (game_state == GAME_IDLE) {
        for (int i = 0; i < NUM_PULSES; i++) {
```

```
            if (c == PULSES[i].symbol) {
                PlayPulse(i);

                char buf[128];
                snprintf(buf, sizeof(buf),
                         "\r\nPulse '%c': Color=%s, Brightness=%d%
%, Frequency=%d Hz\r\n",
                         c,
                         PULSES[i].green ? "Green" :
(PULSES[i].yellow ? "Yellow" : "Red"),
                         PULSES[i].green + PULSES[i].yellow +
PULSES[i].red,
                         PULSES[i].freq);
                UART_SendString(buf);
                return;
            }
        }

        if (c == '+') {
            current_speed = (current_speed + 1) % 3;
            const char* speed_names[] = {"SLOW", "MEDIUM", "FAST"};
            char buf[64];
            snprintf(buf, sizeof(buf), "\r\nSpeed: %s (%d ms)\r\n",
                     speed_names[current_speed],
                     SPEED_DURATIONS[current_speed]);
            UART_SendString(buf);
        } else if (c == 'a' || c == 'A') {
            play_mode = (play_mode + 1) % 3;
            const char* mode_names[] = {"LED + SOUND", "LED ONLY",
"SOUND ONLY"};
            char buf[64];
            snprintf(buf, sizeof(buf), "\r\nPlay mode: %s\r\n",
mode_names[play_mode]);
            UART_SendString(buf);
        } else if (c == '\r' || c == '\n') {
            StartGame();
        }
    } else if (game_state == GAME_PLAYING) {
        if (input_received) return;

        uint8_t expected = sequence[sequence_pos];
        if (c == PULSES[expected].symbol) {
            trace[sequence_pos] = '+';
            score += 10 * (current_speed + 1);
        } else {
            trace[sequence_pos] = '-';
```

```
            mistakes++;
        }
        input_received = 1;
    } else if (c == '\r' || c == '\n') {
        if (game_state == GAME_COUNTDOWN || game_state ==
GAME_PLAYING) {
            UART_SendString("\r\n=== GAME STOPPED ===\r\n");
            StopGame();
        }
    }
}

static void UART_SendString(const char* str) {
    HAL_UART_Transmit(&huart6, (uint8_t*)str, strlen(str),
HAL_MAX_DELAY);
}

int main(void)
{
  HAL_Init();

  SystemClock_Config();

  MX_GPIO_Init();
  MX_TIM1_Init();
  MX_TIM6_Init();
  MX_USART6_UART_Init();
  MX_I2C1_Init();
  MX_TIM4_Init();
  /* USER CODE BEGIN 2 */

  UART_SendString("\r\n\r\n=== RHYTHM GAME ===\r\n");
  UART_SendString("System initialized successfully!\r\n");

  char freq_buf[128];
  snprintf(freq_buf, sizeof(freq_buf), "SYSCLK: %lu Hz\r\n",
HAL_RCC_GetSysClockFreq());
  UART_SendString(freq_buf);
  snprintf(freq_buf, sizeof(freq_buf), "HCLK: %lu Hz\r\n",
HAL_RCC_GetHCLKFreq());
  UART_SendString(freq_buf);
  snprintf(freq_buf, sizeof(freq_buf), "PCLK1: %lu Hz\r\n",
HAL_RCC_GetPCLK1Freq());
  UART_SendString(freq_buf);
  snprintf(freq_buf, sizeof(freq_buf), "PCLK2: %lu Hz\r\n\r\n",
HAL_RCC_GetPCLK2Freq());
```

```
  UART_SendString(freq_buf);

  UART_SendString("=== TESTING BUZZER ===\r\n");

  uint32_t test_arr = (144000000 / 440) - 1;
  __HAL_TIM_SET_AUTORELOAD(&htim1, test_arr);
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, test_arr / 2);
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
  HAL_Delay(500);
  HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);

  HAL_Delay(200);

  test_arr = (144000000 / 523) - 1;
  __HAL_TIM_SET_AUTORELOAD(&htim1, test_arr);
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, test_arr / 2);
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
  HAL_Delay(500);
  HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);

  UART_SendString("Buzzer test complete!\r\n\r\n");

  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_2);
  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_3);
  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);

  SetPWM(0, 0, 0);

  HAL_UART_Receive_IT(&huart6, &uart_rx_byte, 1);

  UART_SendString("Commands:\r\n");
  UART_SendString("  1-9 : Test pulses (color + sound)\r\n");
  UART_SendString("  +   : Change speed (SLOW/MEDIUM/FAST)\r\n");
  UART_SendString("  a   : Change mode (LED+SOUND/LED ONLY/SOUND
ONLY)\r\n");
  UART_SendString("  Enter: Start/Stop game\r\n");
  UART_SendString("\r\nReady! Press 1-9 to test...\r\n");

  while (1)
  {
    uint32_t now = HAL_GetTick();

    if (game_state == GAME_COUNTDOWN) {
        if (now - countdown_start >= 3000) {
            game_state = GAME_PLAYING;
            sequence_pos = 0;
```

```
            pulse_start = now;
            input_received = 0;
            PlayPulse(sequence[0]);

            __HAL_TIM_SET_COUNTER(&htim6, 0);
            __HAL_TIM_SET_AUTORELOAD(&htim6,
SPEED_DURATIONS[current_speed] - 1);
            HAL_TIM_Base_Start_IT(&htim6);

            UART_SendString("=== GO! ===\r\n");
        }
    }

    if (uart_rx_head != uart_rx_tail) {
        char c = uart_rx_buffer[uart_rx_tail];
        uart_rx_tail = (uart_rx_tail + 1) % 128;
        ProcessInput(c);
    }
  }
}
```

3. Вывод

В ходе выполнения работы мы разобрались с принципами работы таймеров, откуда мы создали игру ритма на стенде STM-1.1M. Мы рассматривали конкретно виды таймеров, настройки их частот, и других основных понятии, связанных с ними.